

Introducción a C++ para Resolución de Problemas

TTPS: Técnicas y Estrategias para la Resolución de Problemas

Taller de Tecnologías de Producción de Software

Facultad de Informática
Universidad Nacional de La Plata

23 de agosto de 2026



Facultad de
INFORMÁTICA



UNIVERSIDAD
NACIONAL
DE LA PLATA



Instalar C++ en Windows y Ubuntu

Windows: MSYS2 + GCC

Descargar MSYS2 desde msys2.org, abrir **MSYS2 UCRT64** y ejecutar:

```
pacman -Syu
pacman -S --needed mingw-w64-ucrt-x86_64-gcc
```

Ubuntu: GCC

Abrir una terminal e instalar el compilador y las herramientas básicas:

```
sudo apt update
sudo apt install build-essential
g++ --version
```

Compilar y ejecutar un archivo .cpp

Desde la terminal

Ubicarse en la carpeta que contiene el archivo, por ejemplo programa.cpp.

```
g++ -std=c++17 -Wall -Wextra programa.cpp -o programa  
./programa
```

En Windows, desde MSYS2 UCRT64, el ejecutable se genera como programa.exe y se ejecuta con ./programa.exe.

Qué significa cada opción

- -std=c++17: utiliza el estándar C++17.
- -Wall -Wextra: muestra advertencias útiles del compilador.
- -o programa: define el nombre del ejecutable generado.

Compilar y ejecutar en un solo paso: `g++ programa.cpp -o programa && ./programa`

Plantilla de programa C++

Estructura Básica (.cpp)

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    // Optimización de E/S
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    // Tu código aquí

    return 0;
}
```

Optimización de E/S

En programación competitiva, la velocidad de entrada/salida es crucial.

- `sync_with_stdio(false)`: Desactiva la sincronización entre los streams de C y C++. Evita mezclar `printf/scanf` con `cin/cout`.
- `cin.tie(NULL)`: Desvincula `cin` de `cout`, evitando que se limpie el buffer de salida cada vez que se pide una entrada.

Tipos de variables en C++

Elegir el tipo adecuado

El tipo de una variable determina qué valores puede almacenar y cuánta memoria utiliza.

Tipo	Uso principal
int	Enteros de 32 bits; aproximadamente entre $-2 \cdot 10^9$ y $2 \cdot 10^9$.
long long	Enteros de 64 bits; aproximadamente entre $-4 \cdot 10^{18}$ y $4 \cdot 10^{18}$.
double	Reales de doble precisión, con unas 15 cifras decimales significativas.
long double	Reales con más precisión que double.
char	Un único carácter, como 'A' o '7'.
string	Cadena de caracteres, como "Hola"; requiere incluir <string>.

```
int cantidad = 100;
long long poblacion = 8000000000LL;
double promedio = 8.5;
long double precision = 3.141592653589793L;
char inicial = 'M';
string nombre = "Ada";
```

Lectura y escritura de datos estándar

Lectura con cin

cin recibe datos desde la entrada estándar, normalmente el teclado.

```
int edad;
double altura;
string nombre;

cin >> edad >> altura;
cin >> nombre;
```

El operador » ignora espacios y saltos de línea antes de leer.

Leer una línea completa

```
string frase;
getline(cin, frase);
```

getline también conserva los espacios.

Escritura con cout

cout envía datos a la salida estándar, normalmente la pantalla.

```
cout << "Nombre: " << nombre <<
    "\n";
cout << "Edad: " << edad << endl
    ;
```

El operador < permite concatenar texto y variables.

Ejemplo completo

```
int a, b;
cin >> a >> b;
cout << a + b << '\n';
```

Para grandes volúmenes de datos, conviene usar n en lugar de endl.

Estructuras de Control - Condicionales

Sentencias de Bifurcación

Permiten tomar decisiones en base a expresiones booleanas usando `if`, `else if` y `else`.

Ejemplo de Estructura Tradicional

```
int x = 10;
if (x > 10) {
    cout << "Mayor";
} else if (x == 10) {
    cout << "Igual";
} else {
    cout << "Menor";
}
```

Operador Ternario

Sintaxis compacta para asignación condicional:

`condicion ? valor_si_true :`
`valor_si_false;`

Ejemplo:

```
int a = 5, b = 8;
int mayor = (a > b) ? a : b;
// mayor tomara el valor 8
```

Bucles For

For tradicional:

```
for (int i = 0; i < 5; i++) {  
    cout << i << " ";  
}
```

For basado en rangos (C++11):

```
vector<int> v = {1, 2, 3};  
for (auto x : v) {  
    cout << x << " ";  
}
```

Bucles While y Do-While

While (evalúa al inicio):

```
int x = 3;  
while (x > 0) {  
    cout << x-- << " ";  
}
```

Do-While (evalúa al final, ejecuta al menos una vez):

```
int y = 0;  
do {  
    cout << y << " ";  
} while (y > 0);
```


Estructuras de Datos - STL: Vectores

Standard Template Library (STL)

Biblioteca de plantillas de C++ que proporciona contenedores y algoritmos listos para usar de manera eficiente.

std::vector (Arreglo Dinámico)

Contenedor secuencial que cambia de tamaño dinámicamente. Almacena elementos contiguos en memoria.

```
#include <vector>
...
vector<int> v; // Declaracion vacia
v.push_back(10); // Agrega 10 al final: [10]
v.push_back(20); // Agrega 20 al final: [10, 20]

cout << "Tamaño: " << v.size() << endl; // Imprime 2
cout << "Primer elemento: " << v[0] << endl; // Acceso O(1)

v.pop_back(); // Elimina el ultimo elemento: [20]
```

SGRD
UNL
LADA

Estructuras de Datos - STL: Sets y Maps

std::set (Conjunto Ordenado)

Almacena elementos únicos ordenados de menor a mayor. Implementado internamente como árbol binario balanceado.

```
#include <set>
...
set<int> s;
s.insert(5); // s: {5}
s.insert(2); // s: {2, 5}
s.insert(5); // No duplicados! s:
              {2, 5}

if (s.count(2)) { // Verifica
                  existencia
    cout << "El 2 existe\n";
}
s.erase(2); // Borra el 2: {5}
```

std::map (Diccionario Clave-Valor)

Colección de parejas clave-valor ordenadas por clave.

```
#include <map>
...
map<string, int> m;
m["messi"] = 10;
m["maradona"] = 10;

cout << m["messi"] << "\n"; //
    Acceso

// Si la clave no existe, se crea
// con el valor por defecto (0
// en int)
cout << m["pele"] << "\n"; //
    Imprime 0
```

Estructuras de Datos - STL: Colas

std::queue (Cola FIFO)

Estructura First-In, First-Out. Útil para algoritmos como BFS.

```
#include <queue>
...
queue<int> q;
q.push(1); // Encola 1
q.push(2); // Encola 2
cout << q.front() << "\n"; //
    Imprime 1
q.pop(); // Saca el 1
cout << q.empty() << "\n"; //
    Falso (0)
```

std::priority_queue (Cola de Prioridad)

Ordena elementos por prioridad.
Max-Heap por defecto (mayor al frente).

```
#include <queue>
...
priority_queue<int> pq; // Max-
    Heap
pq.push(10);
pq.push(20);
cout << pq.top() << "\n"; //
    Imprime 20
pq.pop();

// Min-Heap (menor al frente):
priority_queue<int, vector<int>,
    greater<int>> min_pq;
```

Tabla de Complejidades Temporales (Big-O)

Estructura	Acceso	Inserción	Eliminación	Búsqueda
<code>std::vector</code>	$O(1)$	$O(1)^*$	$O(1)^*$	$O(N)$
<code>std::set</code>	—	$O(\log N)$	$O(\log N)$	$O(\log N)$
<code>std::map</code>	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$
<code>std::queue</code>	$O(1)$	$O(1)$	$O(1)$	—
<code>std::priority_queue</code>	$O(1)$	$O(\log N)$	$O(\log N)$	—

* Amortizado (al final del vector).

Recomendaciones Prácticas

- Utilizar `'\n'` en lugar de `endl` para evitar flujos innecesarios de buffer.
- Elegir la estructura de datos adecuada basándose en la complejidad temporal requerida por las restricciones del problema ($N \leq 10^5 \Rightarrow$ evitar soluciones $O(N^2)$).